Module	Problem (input, output, objective)	Candidates (time complexity)	Selection (primary / baseline)	Justification (data property, assumption, when it fails)
A. Relief inventory	n crates, integer weight w_i ≤ 20 kg, value v_i; capacity W (10-70% of Σw). Choose a set maximising Σv with Σw ≤ W (0/1 knapsack, NP-hard, pseudo-polynomial). Our data: n ≤ 1,920, nW ≤ 9.8·10^6.	Exhaustive O(2^n·n); meet-in-the-middle O(2^(n/2)·n); DP over capacity O(nW); branch & bound, worst O(2^n); greedy by v/w O(n log n), no guarantee.	**Primary:** bottom-up DP O(nW) with a keep-table to recover the crates. **Baseline:** exhaustive search (tier S) and ratio greedy (to measure the gap).	Rests on small integer weights, so W ≤ 0.7·20n stays small (largest table 9.8M cells, case 46). Greedy is provably wrong on our data: suboptimal in 15 tier-S cases, 55% of the optimum in trap 47. Fails if weights were large or real-valued (W ~ 10^9): switch to branch & bound or an FPTAS.
B. Route planning	Undirected roads, V ≤ 3,000, E ≤ 3V, minus today's blocked roads; source = HQ. Step u→v costs minutes + penalty(v) ≥ 1, so cost depends on direction. Output: a cheapest route and its cost, or "no route"; the reachable set; a fewest-roads route.	Dijkstra + binary heap O((V+E) log V); Dijkstra with array O(V²); Bellman-Ford O(VE); BFS O(V+E) (reachability and fewest roads only).	**Primary:** Dijkstra with our own binary heap (lazy deletion), each road relaxed in both directions with its own cost; BFS for the reachable set and fewest-roads route. **Baseline:** Bellman-Ford, and the cost of the BFS route.	All step costs are positive (minutes ≥ 1, penalty ≥ 0), so Dijkstra is exact. Graphs are sparse (2E/V = 1.8-5.1), so the heap beats O(V²). Trap 48 (fewest-roads route 4.9× dearer) shows BFS cannot replace Dijkstra. Fails with negative costs (use Bellman-Ford) or very dense graphs (array version).
C. Rescue scheduling	n ≤ 3,000 windows [s_i, f_i) with value v_i. Choose pairwise compatible windows (f_i ≤ s_j, so touching is allowed) maximising (a) the count, (b) Σv.	Exhaustive O(2^n·n); earliest-finish greedy O(n log n); weighted-interval DP with binary search O(n log n); the same DP without binary search O(n²).	**Primary:** (a) earliest-finish greedy; (b) DP over windows sorted by finish, with p(j) found by binary search. **Baseline:** exhaustive search (tier S), O(n²) DP, greedy by value.	(a) Greedy is optimal by an exchange argument, but fails for (b), where one long valuable window beats several short ones; hence DP. L cases have 1,000-1,600 windows; in case 76-44 each overlaps 171 others on average, so O(n²) is slow. Every case has touching pairs, so the test must be f ≤ s, not f < s.
D. Emergency queue	n ≤ 5,000 alarms (tick, priority 1-10). At tick t the alarms with tick ≤ t are present; release one with the highest priority (ties: earlier tick, then smaller alarm_id), continuing past the last tick until empty. Output: the release order.	Unsorted list + linear scan O(n²); sorted-list insertion O(n²); binary max-heap O(n log n); bucket queue of 10 FIFO lists O(n + 10T).	**Primary:** our own binary heap keyed (−priority, tick, alarm_id). **Baseline:** linear scan; bucket queue as a second comparison.	Arrivals of 1.2-1.8 per tick against 1 release leave a peak backlog of 0.2-0.5n (441-1,208 alarms in L/X), so scanning is quadratic. The composite key makes ties deterministic; in study 2 about half of all alarms have priority 9-10. The bucket queue relies on there being only 10 levels; the heap does not. Fails if priorities change while waiting (needs an indexed heap with decrease-key).
E. Radio logs	Log of n ≤ 100,000 symbols, σ = 38. (a) Every start of a pattern of length m ≤ 40, overlaps included. (b) Minimum insertions, deletions and replacements between reports of length a, b ≤ 1,000, with an edit script. (c) Prefix-free binary code minimising Σ freq·length.	(a) naive O(nm); Rabin-Karp O(n+m) expected, O(nm) worst; KMP O(n+m); Horspool O(nm) worst. (b) plain recursion O(3^(a+b)); Wagner-Fischer O(ab); Hirschberg O(ab) time, linear space. (c) fixed 6-bit code; Shannon-Fano (not optimal); Huffman O(n + σ log σ).	**Primary:** KMP; Wagner-Fischer full table plus traceback; Huffman with our own min-heap. **Baseline:** naive search; brute-force recursion (tier S); fixed 6-bit code.	In trap 49 the pattern prefixes match at about 95% of positions, so naive search needs about n(m−1) ≈ 2·10^6 comparisons; KMP never re-reads the log. a·b ≤ 10^6 cells fits in memory. Skewed logs (filler 31-44%, trap 95%) make Huffman pay off: 1.3-3.4 bits/symbol vs 6. Fails if reports were about 10^5 long (table too big: use Hirschberg). A one-symbol log needs a special 1-bit code.
F. Network planning	After closures, the component reachable from the HQ (V' ≤ V, E' ≤ E), cable 1-100. Choose roads linking all of V' with minimum Σ cable.	Kruskal + union-find O(E log E), or O(E α(V)) with counting sort; Prim + binary heap O(E log V); Prim with array O(V²); Borůvka O(E log V).	**Primary:** BFS from the HQ, then Kruskal with counting sort and union-find (path compression, union by rank). **Baseline:** Prim with a heap (equal totals give a cross-check).	Integer cables of 1-100 make the sort linear, and the graphs are sparse. Closures can disconnect the network (study 5), and Kruskal handles forests naturally. Assumes cable cost is the same in both directions. Slower on dense graphs, where Prim with an array (O(V²)) wins.
G. Chains and cuts	(a) Matrices A_1..A_n, A_i is p_(i−1)×p_i, n ≤ 300: the bracketing with the fewest scalar multiplications. (b) Drum of length L ≤ 2,000, price p_j for j ≤ L: the cuts maximising Σ price.	(a) all bracketings Θ(Catalan(n−1)); interval DP O(n³) time, O(n²) space; greedy cheapest pair first (no guarantee); Hu-Shing O(n log n). (b) all 2^(L−1) cut sets; DP O(L²); greedy best price per unit (no guarantee).	**Primary:** interval DP with a split table; bottom-up rod DP with a first-cut table. **Baseline:** exhaustive search on tier S (Catalan(7) = 429 bracketings, p(14) = 135 partitions) and the greedy rules.	n ≤ 300 gives about n³/6 = 4.5·10^6 steps; L ≤ 2,000 gives 2·10^6. Narrow 2-10 stages make the bracketing matter. Premium lengths and prices saturating at 500 make greedy wrong. Fails near n ~ 10^4 (O(n³) too slow: use Hu-Shing).
H. Hard planning questions	(a) 3-CNF with N ≤ 50 switches, m ≤ 6N rules: a satisfying assignment or none. (b) q ≤ 100 values ≤ 1,000, target T: a subset summing exactly to T, or none. (c) Shortest closed Manhattan tour of n ≤ 500 sites. (d) A vertex cover of ≤ k nodes in a graph of n ≤ 400 nodes, or none.	(a) 2^N·m enumeration; DPLL with unit propagation; WalkSAT (cannot prove "none"). (b) 2^q enumeration; meet-in-the-middle O(2^(q/2)·q); DP O(qT). (c) n! enumeration; Held-Karp O(n²·2^n); nearest neighbour O(n²) + 2-opt. (d) C(n,k)·m enumeration; Buss kernel + bounded search tree O(2^k·m); matching 2-approximation O(m).	**Primary:** DPLL; subset-sum DP with reconstruction; Held-Karp for n ≤ 16, NN + 2-opt above that; Buss kernel + branch and bound. **Baseline:** exhaustive search on tier S; the planted objects as reference answers.	Planted solutions guarantee "yes" answers and give bounds to compare against. T ≤ 18,045 in our data, so qT ≤ 2·10^6. Rule ratios span 2.2-5.8, including the hard region near 4.26 where DPLL slows down. Fails for tier L: Held-Karp (2^320 states) and exact cover (k up to 65) are infeasible, so we report heuristic answers with their gap to the planted tour or cover.
I. Nearest sites and checksums	(a) n ≤ 500 sites with integer coordinates: the pair with minimum squared Euclidean distance. (b) The product of two d-digit decimal numbers built from crate serials (A = serials of odd crate_ids concatenated, B = even ones; d ≈ 4n ≤ 7,680 digits).	(a) all pairs O(n²); divide and conquer O(n log n); grid hashing O(n) expected. (b) schoolbook O(d²); Karatsuba O(d^1.585); FFT O(d log d).	**Primary:** divide and conquer (pre-sorted by x and y, strip check of 7 neighbours); Karatsuba on digit arrays, schoolbook below 32 digits. **Baseline:** all pairs; schoolbook. Python's built-in integer product is used only to verify.	Sites are clustered (80% in Gaussian villages), giving many near-ties; exact integer squared distances avoid float error. d ≤ 7,680 means Karatsuba needs about 1.4·10^6 digit products vs 5.9·10^7 for schoolbook. FFT only wins for much larger d; grid hashing degrades when many points share a cell.

Member	Contribution (Iteration 1 work, and the decision modules proposed for Iteration 2)
Hamza Faisal, 23I-2530-A, section A	**Data modules:** generator framework (seeded random streams per case and module, tier/scenario logic, the workbook writer and the content digest); crates and capacity, including traps 46 and 47; road network (geometric networks, the landslide corridor, trap 48, closure waves, target selection). **Validator:** V1-V4 (sheets, headers, types, ordering, ranges, identifiers and references, connectivity, waves). **Claims:** cargo_opt. **Approach note:** part 1 (story to numbers). **Proposes for Iteration 2:** A (relief inventory), B (route planning), F (network planning).
Ahmed Hassan, 24I-2560-B, section B	**Data modules:** radio log per case study, patterns and counts, reports and edit_bound, trap 49; rescue windows; incidents (rates, aftershock bursts). **Validator:** V5 (pattern counts with KMP, edit_bound, log parts, tier-S claims) and V8 (trap rules). **Claims:** window_count_opt, window_value_opt. **Approach note:** part 3 (traps). **Proposes for Iteration 2:** C (rescue scheduling), D (emergency queue), E (radio logs).
Zeyan Ahmed, 24I-2627-B, section B	**Data modules:** relief sites and the planted tour (nearest neighbour + 2-opt); pipeline dimensions and drum prices; sensor graph and planted cover; planted 3-SAT safety rules; planted rations. **Validator:** V6 (planted objects) and V7 (signature and variety rules); the broken test copies and validation_report.txt. **Claims:** drum_opt, tour_opt, cover_min. **Approach note:** parts 2 and 4. **Proposes for Iteration 2:** G (chains and cuts), H (hard planning questions), I (nearest sites and checksums).